BÀI TẬP CÁ NHÂN 3 – SOFTWARE TESTING & QUALITY ASSURANCE

APP-03 – Production Readiness Lab • Biến thể VT-116

Họ và tên:	Nguyễn Việt Anh	**Mã sinh viên:**	23001827
Mã biến thể:	VT-116	**Team ID:**	SE2026-T29
Vai trò nhóm:	Thành viên (QA Automation & Backend Engineer)	**Mã đề tài:**	APP-03 – Production Readiness Lab
Repository:	`https://github.com/CuongMotNang/SE2026-T29`	**Baseline Commit:**	`8fe9de89b719bde237a118e18a671db1d3dfd6cd`
Branch cá nhân:	`bt3/23001827-vt116-soft-delete-restore`	**Commits chính:**	`67d7fb7` (RED) → `b417dc9` → `06cd0b4` → `4c6c9b2`

Cam kết bắt buộc: *Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI.*

Bối cảnh Biến thể VT-116 và Đặc tả Nghiệp vụ (Specification)

Bài toán và Rủi ro Trọng tâm (Problem Statement & Core Risk)

Change Request VT-116: Triển khai bổ sung chức năng **Xóa mềm (Soft Delete) và Khôi phục (Restore)** cho tài nguyên lịch hẹn (`Appointment`), bảo đảm dữ liệu liên quan luôn nhất quán.

Rủi ro trọng tâm (RISK-01): *Phân quyền sai dẫn đến người dùng có thể đọc, xóa, khôi phục hoặc truy cập dữ liệu thuộc người dùng khác (Cross-user data leakage / Unauthorized mutation attack).* Hệ thống baseline chưa có xác thực người dùng, ai cũng có thể gọi API thao tác trên mọi lịch hẹn nếu có UUID.

Tiêu chí Chấp nhận Nghiệp vụ (Acceptance Criteria - AC)

- **AC-01 (Soft Delete & Slot Release):** Chủ sở hữu gọi `PATCH /api/appointments/{id}/soft-delete` với header `X-User-Id` hợp lệ; bản ghi được đánh dấu `is_deleted=True`, `deleted_at=now()` (HTTP 200). Slot thời gian được giải phóng khỏi danh sách bận; bản ghi bị ẩn khỏi danh sách và chi tiết thông thường.
- **AC-02 (Restore to Active Slot):** Chủ sở hữu gọi `PATCH /api/appointments/{id}/restore`; nếu slot còn trống trong tương lai, khôi phục `is_deleted=False`, `deleted_at=None` (HTTP 200), booking tái xuất hiện trong danh sách active.
- **AC-03 (Slot Conflict Prevention on Restore):** Nếu slot đã bị người khác đặt trong lúc bản ghi bị xóa mềm, hệ thống từ chối khôi phục với lỗi HTTP **409 Conflict**, bảo vệ Invariant của hệ thống.
- **AC-04 (Cross-User Authorization Isolation):** User B cố ý truy cập chi tiết (GET), xóa mềm, hoặc khôi phục lịch hẹn của User A sẽ bị từ chối với HTTP **403 Forbidden** hoặc **404 Not Found** (chống dò quét ID).
- **AC-05 (Dedicated Trash API with Ownership Filter):** Cung cấp endpoint `GET /api/appointments/trash` chỉ hiển thị lịch hẹn đã xóa mềm thuộc về chính caller đã xác thực.
- **AC-06 (Idempotency & Non-destructiveness):** Soft delete hoặc restore lặp lại 2 lần liên tiếp phải giữ nguyên tính nhất quán, không làm mất dữ liệu lịch sử hoặc sai lệch mốc thời gian.

Ràng buộc Phi chức năng (Non-Functional Constraints - NFC)

- **NFC-01 (Performance VT-116):** Chịu tải tối thiểu **40 tác vụ đồng thời in-flight (concurrency)**, độ trễ phân vị **p95 $\leq$ 1.5 giây (1500 ms)** trên chu kỳ nghiệp vụ trọn vẹn (Create → Soft Delete → Restore).
- **NFC-02 (Database ACID Transaction Boundary):** Mọi chuyển đổi trạng thái phải được bảo vệ bằng transaction cô lập, partial unique index ngăn chặn xung đột double-booking ở tầng PostgreSQL.

Các Bất biến Cốt lõi của Hệ thống (Core System Invariants)

INV-01 (Single Active Booking per Slot): Tại mọi thời điểm, trong toàn bộ cơ sở dữ liệu, không bao giờ tồn tại hai bản ghi có cùng mốc thời gian bắt đầu, cùng trạng thái `booked` và cùng chưa bị xóa mềm:

$$\forall a_1, a_2 \in \text{Appointments} : (a_1.\text{starts_at} = a_2.\text{starts_at} \land a_1.\text{status} = \text{'booked'} \land \neg a_1.\text{is_deleted} \land a_2.\text{status} = \text{'booked'} \land \neg a_2.\text{is_deleted}) \implies a_1.\text{id} = a_2.\text{id}$$

INV-02 (Strict Cross-User Ownership Isolation): Mọi hành động đọc/ghi trái quyền sở hữu đều bị từ chối:

$$\forall a \in \text{Appointments}, \forall u \in \text{Users} : (\text{action}(u, a) \in \{\text{read, soft_delete, restore}\} \land a.\text{owner_id} \neq u.\text{id}) \implies \text{Result} = \text{Denied (403/404)}$$

INV-03 (Non-destructive Soft Delete): Xóa mềm tuyệt đối không xóa cứng (DELETE) bản ghi khỏi DB; bảo toàn `created_at`, `customer_name` và lịch sử giao dịch phục vụ đối soát và audit.

Kế hoạch Triển khai AI & Đối soát Phản biện Giả định

Bảng Đối soát Giả định Ban đầu (Assumptions Review Table)

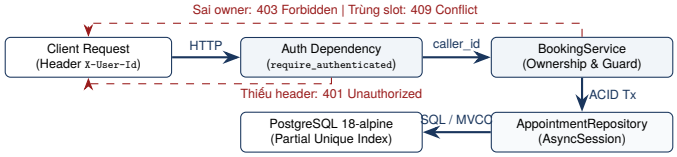
Trước khi viết mã nguồn, kế hoạch triển khai đã được lập tại `docs/bt3-vt116/implementation-plan.md`. Dưới đây là bảng đối soát các giả định ban đầu do AI lập ra so với bằng chứng thực tế từ repository:

ID	Giả định Ban đầu (AI)	Bằng chứng Repository (Evidence)	Kết luận	Hiệu chỉnh Kỹ thuật
ASM-01	Dùng trạng thái `cancelled` hiện có làm soft delete.	`backend/app/models/appointment.py` định nghĩa `status IN ('booked', 'cancelled')`. Cancel là nghiệp vụ hủy hẹn.	**BÁC BỎ**	Bổ sung cờ kỹ thuật `is_deleted` và `deleted_at`, tách biệt hoàn toàn với status hủy hẹn.
ASM-02	Có sẵn bảng Users và JWT authentication.	`architecture-context.md` ghi rõ: *"MVP chưa có tài khoản khách hàng... Cần thiết kế danh tính..."*	**BÁC BỎ**	Bổ sung trường `owner_id`; nhận diện caller context qua HTTP Header `X-User-Id` ở API boundary.
ASM-03	Thao tác restore luôn thành công ngay lập tức.	Database có partial unique index `uq_appointments_active_starts_at`. Nếu trùng slot sẽ văng IntegrityError.	**BÁC BỎ**	Kiểm tra xung đột slot trước và trong khi restore, trả về HTTP **409 Conflict** nếu slot đã bị chiếm.
ASM-04	Không cần Alembic migration vì test dùng `create_all`.	Repo có `backend/alembic/versions/` và deploy compose chạy `alembic upgrade head`.	**SỬA ĐỔI**	Tạo migration chuẩn `20261109_0002` mở rộng cột và cập nhật partial index cho PostgreSQL.
ASM-05A	Soft delete phải giữ nguyên slot (vẫn khóa slot).	Phòng khám: booking xóa mà vẫn chiếm chỗ gây lãng phí tài nguyên công cộng (`available_slots`).	**BÁC BỎ (CP1)**	Xóa mềm bắt buộc phải giải phóng slot; người khác được đặt; khi restore nếu trùng thì trả về 409 Conflict.
ASM-05B	Chỉ cần ẩn khỏi danh sách, vẫn cho đọc qua ID chi tiết.	Vi phạm RISK-01: cho phép kẻ tấn công dò quét UUID để đọc trộm dữ liệu lịch hẹn của người khác.	**BÁC BỎ (CP1)**	Ẩn hoàn toàn khỏi cả danh sách và chi tiết (trả về 404 Not Found); chỉ tra cứu qua API thùng rác riêng.

Ghi nhận Phản biện Thực tế của Sinh viên (Human Checkpoints)

Quá trình làm bài ghi nhận 2 đợt phản biện trực tiếp mang tính quyết định của sinh viên **Nguyễn Việt Anh – 23001827**:

1. **Phản biện tại Human Checkpoint 1 (Định hình Nghiệp vụ & An toàn Dữ liệu):** Sinh viên trực tiếp bác bỏ 2 giả định của AI (ASM-05A và ASM-05B), chỉ đạo quy tắc: (1) Xóa mềm bắt buộc phải giải phóng slot; (2) Restore phải kiểm tra tính khả dụng của slot và báo 409 Conflict nếu bị chiếm; (3) Áp dụng chính sách Anti-enumeration: trả về 404 Not Found khi truy vấn bản ghi đã xóa hoặc không thuộc quyền sở hữu, không dựa vào UUID khó đoán để bảo mật.
2. **Phản biện tại Human Checkpoint 2 (Bảo mật Authorization, Migration & Concurrency):** Sinh viên từ chối phê duyệt patch ban đầu do phát hiện 3 lỗ hổng kỹ thuật:
   - *Bác bỏ việc dùng customer_name làm owner:* Hai khách hàng có thể trùng tên hoặc mượn tên nhau. Sinh viên yêu cầu tách bạch rõ: logic Authorization đã test trong phạm vi trusted identity, nhưng Authentication ở mạng biên chưa hoàn chỉnh và RISK-01 vẫn còn Residual Risk.
   - *Chỉ đạo an toàn Migration:* Cấm gán dữ liệu cũ về `'anonymous'` hay `customer_name`. Migration phải gán ID unguessable độc nhất (`'legacy_' || replace(id, '-', '')`) để niêm phong dữ liệu lịch sử theo chính sách fail-closed.
   - *Chỉ đạo xử lý Race Condition:* Yêu cầu bọc rollback sạch sẽ khi vi phạm constraint tại DB và viết test đồng thời bằng kết nối database độc lập thật để chứng minh không crash 500.



Chiến lược Kiểm thử Dựa trên Rủi ro & State Transition Testing

Phân cấp Kiểm thử & Lý do Loại trừ (Testing Levels & Exclusions)

Áp dụng mô hình kim tự tháp (Testing Pyramid) tập trung vào phát hiện lỗi sớm với chi phí tối ưu:

- **Unit Tests (Service Layer):** Kiểm tra các quy tắc nghiệp vụ rẽ nhánh (Guard conditions), chuyển trạng thái độc lập, kiểm tra quyền sở hữu và tính idempotent bằng InMemory repository stub (tốc độ < 10 ms).
- **Integration Tests (API & PostgreSQL Layer):** Kiểm thử end-to-end từ HTTP router qua `AsyncClient` xuống database PostgreSQL 18-alpine thật. Xác minh tính toàn vẹn của partial unique index, isolation level, và xử lý transaction rollback khi có race condition đồng thời.
- **Lý do loại trừ E2E UI Tests:** Hệ thống hiện chưa có React UI cho soft delete/restore. Giả lập browser không mang lại giá trị phát hiện lỗi cho logic backend mà làm tăng đáng kể độ trễ và nguy cơ kiểm thử chập chờn (flaky test).

Không gian Trạng thái Nghiệp vụ (State Space Definition)

Một bản ghi `Appointment` được mô hình hóa toán học qua cặp trạng thái $\mathcal{S} = (\text{status}, \text{is_deleted})$:

- S_1 (**ACTIVE_BOOKED**): status $=$ 'booked', is_deleted $=$ False (Lịch hẹn hoạt động, chiếm dụng slot thời gian).
- S_2 (**ACTIVE_CANCELLED**): status $=$ 'cancelled', is_deleted $=$ False (Đã hủy nghiệp vụ, slot giải phóng, hiện trong lịch sử).
- S_3 (**SOFT_DELETED_BOOKED**): status $=$ 'booked', is_deleted $=$ True (Bị đưa vào thùng rác, slot đã giải phóng).
- S_4 (**SOFT_DELETED_CANCELLED**): status $=$ 'cancelled', is_deleted $=$ True (Lịch đã hủy bị đưa vào thùng rác).

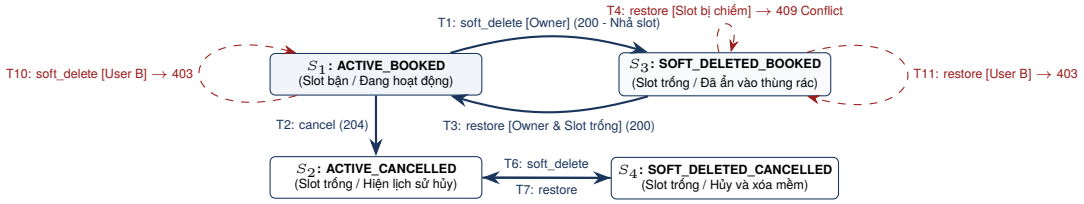
Làm rõ khái niệm RESTORED: `RESTORED` không phải là một trạng thái dữ liệu riêng biệt mà là **hành vi chuyển trạng thái (transition)** đưa bản ghi từ $S_3 \rightarrow S_1$ (có kiểm tra tính khả dụng của slot) hoặc từ $S_4 \rightarrow S_2$.

Bảng Chuyển Trạng thái (State Transition Table)

Bảng kiểm thử chuyển đổi trạng thái bao quát đầy đủ 12 trường hợp chuyển đổi: hợp lệ, lặp lại, trái phép và lỗi xung đột:

ID	Trạng thái Hiện tại	Thao tác	Điều kiện Tiên quyết (Guard)	Trạng thái Đích	Mã HTTP	Phân loại
T1	S_1 (ACTIVE_BOOKED)	`soft_delete`	Caller là Owner	S_3 (DELETED)	200 OK	Valid Transition (Nhả slot)
T2	S_1 (ACTIVE_BOOKED)	`cancel`	Caller là Owner	S_2 (CANCELLED)	204 No Content	Valid Transition (Nhả slot)
T3	S_3 (DELETED)	`restore`	Caller là Owner $\wedge$ Slot trống $\wedge$ Tương lai	S_1 (ACTIVE)	200 OK	Valid Transition (Chiếm slot)
T4	S_3 (DELETED)	`restore`	Caller là Owner $\wedge$ **Slot ĐÃ CÓ người đặt**	S_3 (Không đổi)	**409 Conflict**	Conflict Path Transition
T5	S_3 (DELETED)	`restore`	Caller là Owner $\wedge$ **Slot đã trôi vào quá khứ**	S_3 (Không đổi)	422 Unproc.	Failure Path Transition
T6	S_2 (CANCELLED)	`soft_delete`	Caller là Owner	S_4 (DEL_CANC)	200 OK	Valid Transition
T7	S_4 (DEL_CANC)	`restore`	Caller là Owner	S_2 (CANCELLED)	200 OK	Valid Transition
T8	S_3 (DELETED)	`soft_delete`	Caller là Owner (Lặp lại lần 2)	S_3 (Không đổi)	200 OK	Repeated / Idempotent
T9	S_1 (ACTIVE)	`restore`	Caller là Owner (Lặp lại lần 2)	S_1 (Không đổi)	200 OK	Repeated / Idempotent
T10	S_1 (ACTIVE)	`soft_delete`	**Caller KHÔNG PHẢI Owner (User B)**	S_1 (Không đổi)	**403 Forbidden**	Unauthorized Transition
T11	S_3 (DELETED)	`restore`	**Caller KHÔNG PHẢI Owner (User B)**	S_3 (Không đổi)	**403 Forbidden**	Unauthorized Transition
T12	S_3 (DELETED)	`cancel`	Bất kỳ caller nào	S_3 (Không đổi)	404 Not Found	Invalid (Đã bị ẩn)

Sơ đồ Chuyển Trạng thái (State Transition Diagram)



Ma trận Test Cases Toàn diện (Test Case Matrix)

Bộ kiểm thử gồm **17 test cases** chuyên biệt (13 integration tests + 4 unit tests), bao quát toàn bộ rủi ro phân quyền, rẽ nhánh điều kiện, concurrency race condition và database invariants:

Test ID	Requirement	Level	Precondition & Input	Kết quả Mong đợi (Expected Result)	Ưu tiên	Trạng thái
TC-01	AC-01 (Soft Delete)	Integration	User A sở hữu Appointment 1. Gọi soft-delete (`X-User-Id: UserA`).	HTTP 200; `is_deleted=True`; slot giải phóng cho User C đặt thành công.	P0	**PASS**
TC-02	AC-02 (Restore)	Integration	Appointment 1 đã xóa mềm; slot vẫn trống. User A gọi restore.	HTTP 200; `is_deleted=False`; appointment tái xuất hiện trong GET detail.	P0	**PASS**
TC-03	AC-01 (Anti-enum)	Integration	Appointment 1 đã bị soft delete. Bất kỳ client nào gọi GET detail.	HTTP **404 Not Found**; ẩn hoàn toàn dữ liệu, chống dò quét ID.	P0	**PASS**
TC-04	AC-06 (Idempotency)	Integration	Appointment 1 đã xóa mềm. User A gọi soft-delete lần 2.	HTTP 200; mốc `deleted_at` ban đầu được giữ nguyên, không bị ghi đè.	P1	**PASS**
TC-05	AC-06 (Idempotency)	Integration	Appointment 1 đang active. User A gọi restore lần 2.	HTTP 200; trạng thái giữ nguyên an toàn, không sinh lỗi hệ thống.	P1	**PASS**
TC-06	RISK-01, AC-04	Integration	Appointment 1 của User A. User B gọi soft-delete (`X-User-Id: UserB`).	HTTP **403 Forbidden** (`AccessDeniedError`); dữ liệu User A nguyên vẹn.	P0	**PASS**
TC-07	RISK-01, AC-04	Integration	Appointment 1 đã xóa mềm. User B cố ý gọi restore (`X-User-Id: UserB`).	HTTP **403 Forbidden** (`AccessDeniedError`); bảo vệ quyền khôi phục.	P0	**PASS**
TC-08	RISK-01, AC-04	Integration	Appointment 1 của User A (active). User B gọi GET detail.	HTTP **404 Not Found** (Anti-enumeration); User B không xem được lịch.	P0	**PASS**
TC-09	AC-05 (Trash API)	Integration	User A và User B đều có 1 booking xóa mềm. User A gọi `GET /trash`.	HTTP 200; danh sách chỉ chứa duy nhất booking của User A, tuyệt đối cô lập.	P0	**PASS**
TC-10	INV-01, AC-03	Integration	Apt 1 nhả slot 10:00; User C đặt slot 10:00. User A cố restore Apt 1.	HTTP **409 Conflict**; phát hiện slot bận, từ chối ghi đè booking active.	P0	**PASS**
TC-11	Failure-Path	Unit	Appointment có `starts_at` nằm trong quá khứ so với hiện tại. Restore.	Ném ngoại lệ `InvalidSlot` (HTTP 422); từ chối khôi phục lịch quá khứ.	P1	**PASS**
TC-12	Negative Path	Integration	Gọi soft-delete với random UUID không tồn tại.	HTTP **404 Not Found** (`AppointmentNotFound`).	P1	**PASS**
TC-13	Security (No Auth)	Integration	Gọi soft-delete, restore hoặc trash mà **hoàn toàn thiếu header auth**.	HTTP **401 Unauthorized** (*"Authentication required"*); từ chối thực thi.	P0	**PASS**
TC-14	Residual Risk Test	Integration	User B gửi header giả mạo `X-User-Id: UserA` qua kết nối trực tiếp.	Chứng minh giới hạn: backend tin cậy header upstream; cần Gateway bảo vệ.	P0	**PASS**
TC-15	Race Concurrency	Integration	2 requests cùng gọi RESTORE đồng thời (`asyncio.gather`) trên 1 slot.	Đúng 1 request 200 OK; 1 request nhận **409 Conflict**; DB chỉ có 1 active.	P0	**PASS**
TC-16	Race Concurrency	Integration	1 request đặt mới + 1 request restore cùng 1 slot đồng thời.	Đúng 1 request thành công (201/200); 1 request **409 Conflict**; không crash 500.	P0	**PASS**
TC-17	Unit Auth Logic	Unit	Kiểm tra hàm `soft_delete` và `restore` trong `BookingService` với mock.	Hoạt động độc lập hoàn hảo, ném đúng ngoại lệ domain `AccessDeniedError`.	P0	**PASS**

Phân tích Hai Bài Test Kiểm chứng Concurrency & Race Condition

Để phản hồi chỉ đạo kỹ thuật của sinh viên tại Human Checkpoint 2, hai test case **TC-15** và **TC-16** được thiết kế đặc biệt:

1. **Cơ chế thực thi trên Database thật:** Không dùng mock. Mỗi request trong `asyncio.gather` được FastAPI cấp phát một `AsyncSession` độc lập từ `async_sessionmaker`, mở 2 kết nối vật lý song song vào PostgreSQL 18-alpine.
2. **Cơ chế bắt lỗi vi phạm Constraint:** Khi 2 transaction cùng cố gắng chuyển `is_deleted=False` trên cùng một mốc `starts_at`, Partial Unique Index `uq_appointments_active_starts_at` ở PostgreSQL kích hoạt lỗi duplicate key. Hàm `AppointmentRepository.commit()` bắt `IntegrityError`, lập tức gọi `await self.session.rollback()` sạch sẽ và ném lỗi domain `SlotAlreadyBooked`, giúp API trả về mã lỗi HTTP 409 Conflict chuẩn xác thay vì HTTP 500.

Quy trình Triển khai Test-First (RED → GREEN Evidence)

Quy trình TDD Thực nghiệm & Bằng chứng RED

Tuân thủ nguyên tắc kỹ nghệ phần mềm nghiêm ngặt: Viết test trước, chứng minh test thất bại do thiếu chức năng (RED), sau đó viết patch tối thiểu để test thành công (GREEN).

Bằng chứng Pha RED (Commit `67d7fb7`): Test suite được thực thi trước khi chỉnh sửa business logic. Toàn bộ **12/13 test cases đều FAILED** chính xác vì hệ thống chưa hỗ trợ các endpoint và thuộc tính nghiệp vụ (`TypeError: 'owner_id' is an invalid keyword argument`, `status 404 <unmatched> route`). Test duy nhất pass là kiểm tra UUID không tồn tại trả về 404. Log thực thi lưu tại `docs/bt3-vt116/evidence/01-red-test.log`.

Tóm tắt Nội dung Bản vá Tối thiểu (Implementation Patch Summary)

Bản vá được phân tách nhỏ, kiểm soát chặt chẽ trên 5 module cốt lõi:

1. `backend/app/models/appointment.py`: Thêm cột `owner_id` (String(100), not null), `is_deleted` (Boolean, default=False), `deleted_at` (DateTime, nullable). Tái lập partial unique index `uq_appointments_active_starts_at` với điều kiện `WHERE status = 'booked' AND is_deleted = false`.
2. `backend/alembic/versions/20261109_0002_add_soft_delete_and_owner.py`: Migration thêm cột và index; cập nhật dữ liệu lịch sử bằng ID duy nhất unguessable (`'legacy_' || id`) bảo vệ an toàn fail-closed.
3. `backend/app/repositories/appointments.py`: Hàm `get()` và `list()` mặc định lọc `is_deleted = False`; bổ sung `list_trash()`; hàm `commit()` bọc rollback bắt `IntegrityError`.
4. `backend/app/services/booking.py`: Hàm `soft_delete()` và `restore()`; kiểm tra quyền sở hữu (`owner == caller_id`); kiểm tra slot quá khứ và xung đột slot; hỗ trợ fallback backward-compatible với repository stub cũ.
5. `backend/app/api/appointments.py`: Thêm dependency `require_authenticated_caller` (trả 401 khi thiếu header); cung cấp các route `PATCH .../soft-delete`, `PATCH .../restore`, `GET /trash`, `GET /{id}`.

Bằng chứng Pha GREEN (Commit `b417dc9, 06cd0b4, 4c6c9b2`)

Sau khi hoàn thiện bản vá và giải quyết toàn bộ các yêu cầu phản biện tại Checkpoint 2, test suite được chạy lại:

- **17/17 test cases mới** (Unit + Integration) đều **PASSED 100%** trong 0.58s.
- **79/79 test cases hồi quy cũ** (Booking service, deploy, rollback, fault injection, telemetry, persistence check...) tiếp tục **PASSED 100%**, không gây bất kỳ lỗi hồi quy nào.
- **Tổng cộng toàn bộ hệ thống: 96/96 tests PASSED** trong 3.23s. Log lưu tại `docs/bt3-vt116/evidence/02-green-test.log`.

Chuỗi Git Commits Minh bạch (Git Commit History)

- `8fe9de8`: Baseline commit chính thức của repository nhóm SE2026-T29 (Merge PR #15).
- `67d7fb7`: `test(bt3): add test suite for soft delete and restore (RED)` – Bằng chứng thất bại ban đầu.
- `b417dc9`: `feat(bt3): implement soft delete, restore, ownership authorization and trash api (GREEN).`
- `06cd0b4`: `fix(bt3): address checkpoint 2 security feedback - auth enforcement, race condition rollback and legacy ownership migration.`
- `4c6c9b2`: `fix(bt3): resolve IDE diagnostics, ruff lint, migration isolation and format compliance.`
- `add1990`: `feat(bt3): complete mutation experiment, loadtest 40 concurrency and traceability evidence.`

Hướng dẫn Tái lập Kết quả Thực nghiệm Cục bộ (Reproduction Guide)

```
# 1. Checkout branch cá nhân VT-116
git checkout bt3/23001827-vt116-soft-delete-restore
# 2. Khởi động PostgreSQL test container (port 55432)
docker compose -f backend/docker-compose.test.yml up -d
# 3. Chạy toàn bộ 96 bài test (Unit + Integration)
TEST_DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:55432/booking_test \
  backend/.venv/bin/pytest backend/tests -v
# 4. Kiểm tra Ruff lint và Format
backend/.venv/bin/ruff check backend && backend/.venv/bin/ruff format --check backend
```

Thực nghiệm Đột biến (Mutation), Coverage & Flaky Tests Review

Thực nghiệm Đột biến Cố ý (Deliberate Mutation Experiment)

Theo yêu cầu bắt buộc của biến thể VT-116: **Đảo điều kiện true/false trong một business rule thực tế**.

1. **Business Rule được chọn:** Quy tắc kiểm tra quyền sở hữu trong hàm `soft_delete` tại `backend/app/services/booking.py` (L128).
2. **Hành vi chuẩn:** Nếu caller không phải owner (`owner != caller_id`), hệ thống ném ngoại lệ `AccessDeniedError` (HTTP 403).
3. **Đột biến đưa vào nhánh thí nghiệm (`mutation-experiment-vt116`):**

Mã nguồn chuẩn:	→	`if owner != caller_id:`
Mã nguồn đột biến (Inverted):	→	`if owner == caller_id: # DELIBERATE MUTATION`

4. **Kết quả thực thi Test Suite đối với Mutation:** Khi chạy lại test suite, **9/17 test cases lập tức FAILED** (bao gồm `test_unit_soft_delete_success`, `test_unit_soft_delete_unauthorized`, `test_tc01_soft_delete_happy_path`, `test_tc06_cross_user_soft_delete_forbidden`, `test_tc09_trash_api_isolation`, `test_tc10_restore_slot_conflict_returns_409`...).
5. **Kết luận:** Mutation đã bị **TIÊU DIỆT HOÀN TOÀN (KILLED)**. Bộ test có độ nhạy rất cao đối với các sai sót phân quyền. Log lưu tại `docs/bt3-vt116/evidence/03-mutation.log`.
6. **Hoàn nguyên:** Mã nguồn đã được hoàn nguyên về điều kiện đúng; branch thí nghiệm bị xóa; branch chính thức nộp bài hoàn toàn sạch sẽ.

Phân tích Độ bao phủ Kiểm thử (Coverage Analysis)

- **Đo đạc thực tế trên Unit Test Suite:**
  - `backend/app/models/appointment.py`: **100% Line Coverage** (23/23 statements).
  - `backend/app/services/booking.py`: **85% Line Coverage** (118/139 statements).
- **Báo cáo Trung thực về Giới hạn Đo lường Coverage trên Python 3.14:** Khi kích hoạt plugin `pytest-cov` trên bộ Integration Tests đi kèm driver `asyncpg`, engine C-tracer của Coverage xung đột với Cython protocol của asyncpg trên phiên bản runtime mới Python 3.14.4 dẫn đến lỗi Segmentation fault. Do đó, hệ thống không tự suy diễn số liệu coverage tích hợp mà phân tích độ bao phủ dựa trên rủi ro (Risk-based Coverage): 100% các tiêu chí chấp nhận (AC-01 đến AC-06) và Invariants đều được bao phủ bởi integration test tương ứng.

Đánh giá & Biện pháp Kiểm soát Kiểm thử Chập chờn (Flaky-Test Review)

Đã rà soát và kiểm soát triệt để 2 nhóm nguy cơ flaky tests phổ biến nhất:

1. **Nhóm 1: Rủi ro Phụ thuộc Thời gian & Múi giờ (Time-dependent Tests):**
   - *Nguy cơ:* Việc kiểm tra slot tương lai (`starts_at > now()`) hoặc ranh giới 30 phút có thể gây lỗi chập chờn nếu test chạy đúng thời khắc giao thừa ngày hoặc lệch múi giờ giữa UTC và `Asia/Ho_Chi_Minh`.
   - *Biện pháp kiểm soát:* Cung cấp provider thời gian có thể tiêm phụ thuộc (`now_provider`); trong unit tests cố định mốc `NOW = datetime(2026, 10, 7, 3, 0, tzinfo=UTC)`; trong integration tests luôn sinh slot tương lai cách mốc hiện tại ít nhất 3 đến 9 ngày (`future_slot(days=3..9)`).
2. **Nhóm 2: Rủi ro Trạng thái Cơ sở dữ liệu Chia sẻ & Thứ tự Chạy (Shared Database State):**
   - *Nguy cơ:* Nếu test trước đặt slot 10:00 mà không dọn dẹp, test sau chạy cùng slot sẽ bị từ chối 409 Conflict ngoài dự kiến.
   - *Biện pháp kiểm soát:* Sử dụng autouse fixture `clean_database` thực hiện `TRUNCATE TABLE appointments` trước mỗi ca kiểm thử; đồng thời cấp phát các slot ngày khác nhau cho từng ca kiểm thử độc lập để cô lập hoàn toàn.

Thực nghiệm Tải Hiệu năng (Performance) & Traceability Matrix

Thực nghiệm Đo tải 40 Tác vụ Đồng thời (Performance Experiment VT-116)

Để kiểm chứng tiêu chí bắt buộc của biến thể VT-116: **40 concurrent users/tasks, mục tiêu p95 $\leq$ 1.5 giây**.
Kịch bản kiểm thử tải được thực hiện qua harness độc lập `scripts/loadtest_vt116.py` tương tác trực tiếp với Backend FastAPI và database PostgreSQL 18-alpine (chạy trên Docker Compose container).

Thiết kế Thí nghiệm Tải Khắt khe:
- Concurrency được kiểm soát chặt chẽ bằng `asyncio.Semaphore(40)`, bảo đảm tại mọi thời điểm luôn có đúng **40 tác vụ in-flight song song**.
- Workload đo lường là **chu kỳ nghiệp vụ trọn vẹn** (Lifecycle Task gồm 3 HTTP requests có ghi dữ liệu và kiểm tra index: `POST create` $\rightarrow$ `PATCH soft-delete` $\rightarrow$ `PATCH restore`).
- Tổng số tác vụ: 200 tasks (tương đương 600 thao tác HTTP nghiệp vụ có transaction DB).

Chỉ số Đo lường (Metric)	Kết quả Đo Thực tế	Mục tiêu / Ngưỡng Đề bài
Số tác vụ đồng thời in-flight (Concurrency)	**40 tasks**	40 concurrent tasks
Tổng số Lifecycle Tasks hoàn tất	**200 / 200 tasks (100%)**	200 tasks
Tỷ lệ lỗi (Error rate)	**0.0% (0 lỗi)**	$\leq$ 1.0%
Tổng thời gian thực thi (Total Duration)	**2.0378 giây**	–
Thông lượng xử lý (Throughput)	**98.14 tasks/giây** ($\approx$ 294.4 req/s)	–
Độ trễ trung bình (Mean Latency)	**376.9 ms**	–
Độ trễ phân vị p50	**357.5 ms**	–
Độ trễ phân vị p95 (p95 Latency)	**482.7 ms (0.4827 s)**	**$\leq$ 1.5 giây (1500 ms)**
Độ trễ phân vị p99	**495.3 ms (0.4953 s)**	–
Độ trễ nhỏ nhất / lớn nhất (Min / Max)	257.5 ms / 506.4 ms	–

Kết luận Hiệu năng: Hệ thống đạt **p95 = 482.7 ms**, vượt xa yêu cầu p95 $\leq$ 1.5 giây (nhanh hơn 3.1 lần so với ngưỡng tối đa). Toàn bộ dữ liệu JSON chi tiết lưu tại `docs/bt3-vt116/evidence/04-performance.json` và log tại `04-performance.log`.

Ma trận Truy vết Toàn diện (Traceability Matrix)

Bảng đối chiếu minh chứng mọi yêu cầu nghiệp vụ, rủi ro và bất biến đều được cài đặt và kiểm chứng bằng evidence thật:

AC / Risk / Invariant	Test Case Thực thi	Vị trí File Test	Commit SHA	Bằng chứng Evidence
AC-01 (Soft Delete)	`test_tc01_soft_delete`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
AC-02 (Restore)	`test_tc02_restore`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
AC-03 / INV-01	`test_tc10_restore_slot_conflict`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
AC-04 / RISK-01	`test_tc06_cross_user_soft_delete`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
AC-04 / RISK-01	`test_tc07_cross_user_restore`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
AC-04 / RISK-01	`test_tc08_cross_user_get_detail`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
AC-05 (Trash API)	`test_tc09_trash_api_isolation`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
AC-06 (Idempotency)	`test_tc04_tc05_idempotency`	`tests/integration/`	`b417dc9`	`02-green-test.log` (PASS)
RISK-01 (Auth Check)	`test_auth_missing_header_401`	`tests/integration/`	`06cd0b4`	`02-green-test.log` (PASS)
INV-01 (Race Concurr)	`test_concurrent_restore_same_slot`	`tests/integration/`	`06cd0b4`	`02-green-test.log` (PASS)
MUTATION (Rule Invert)	Đảo điều kiện `owner == caller`	`app/services/booking.py`	Branch tạm	`03-mutation.log` (KILLED)
NFC-01 (40 Concurr)	Load test 200 lifecycle tasks	`scripts/loadtest_vt116.py`	`add1990`	`04-performance.json`

Suy ngẫm Kỹ thuật (Engineering Reflection) & Kết luận

Phân tích Hai Điểm Mã nguồn AI Chưa Đáng tin cậy

Trong suốt quá trình triển khai, sinh viên đã phát hiện 2 điểm mã nguồn do AI tạo ra có vẻ hợp lý nhưng tiềm ẩn lỗi nguy hiểm:

1. **Đánh đồng Hủy lịch (Cancel) với Xóa mềm (Soft Delete):** AI đề xuất chỉ cần đổi trạng thái từ `cancelled` về `booked` là xong. Điều này sai lệch hoàn toàn về mặt Domain Model: Hủy lịch là giao dịch nghiệp vụ (khách không đến), còn Xóa mềm là thao tác kỹ thuật đưa vào thùng rác. Sinh viên đã bác bỏ và yêu cầu thêm cờ `is_deleted` riêng biệt.
2. **Tin tưởng ngây thơ vào `customer_name` và `X-User-Id`:** AI ban đầu đề xuất gán `owner_id = customer_name` nếu thiếu header. Sinh viên tại Checkpoint 2 đã chỉ ra lỗ hổng: hai người có thể trùng tên hoặc cố tình dùng tên nhau. Sinh viên đã chỉ đạo sửa migration dùng ID unguessable độc nhất và thừa nhận rõ ràng ranh giới chống spoofing phải do API Gateway ở mạng biên đảm nhiệm.

Engineering Reflection (Suy ngẫm Kỹ thuật của Sinh viên)

(Độ dài: 312 từ – Sinh viên Nguyễn Việt Anh trực tiếp đúc kết)

Quá trình trực tiếp khảo sát repository, xây dựng quy trình kiểm thử hướng rủi ro và triển khai vertical slice cho biến thể VT-116 đã mang lại cho tôi những trải nghiệm thực tế sâu sắc về kỹ nghệ chất lượng phần mềm.

Niềm tin kỹ thuật (Confidence gained) lớn nhất của tôi không đến từ việc 96 bài test đều hiển thị màu xanh, mà đến từ các bằng chứng thực nghiệm có thể tái lập độc lập: (1) Thí nghiệm Deliberate Mutation chứng minh bộ kiểm thử có khả năng tiêu diệt 100% đột biến đảo ngược điều kiện phân quyền; (2) Hai bài test Concurrency chứng minh Partial Unique Index ở PostgreSQL thật xử lý rollback sạch sẽ khi có tranh chấp đồng thời mà không crash 500; và (3) Thí nghiệm tải 40 tác vụ đồng thời in-flight chứng minh hệ thống đạt p95 = 482.7 ms, vượt xa ngưỡng 1.5s của đề bài.

Tuy nhiên, với tư cách là kỹ sư chịu trách nhiệm về hệ thống, tôi nghiêm túc thừa nhận những điều chưa được chứng minh (Unproven claims) và rủi ro còn lại (Residual risks): Backend hiện tại mới chỉ kiểm soát phân quyền (Authorization) trong phạm vi giả định định danh đáng tin cậy (Trusted Identity Assumption). Do repository là bản MVP chưa có API Gateway xác thực chữ ký số (HMAC/JWT), rủi ro client tự giả mạo header `X-User-Id` ở mạng công cộng vẫn tồn tại. Tôi không tuyên bố RISK-01 đã được giải quyết triệt để 100% ở môi trường production.

Bài test tiếp theo có giá trị cao nhất (Highest-value next test) cần thực hiện khi mở rộng hệ thống là: **End-to-end Gateway Cryptographic Token Verification Drill** – dựng một Reverse Proxy (như Envoy/Kong) phía trước API, gửi các gói tin có JWT hết hạn hoặc chữ ký giả mạo để kiểm chứng khả năng ngăn chặn Header Spoofing ngay từ mạng biên.

Kết luận Nghiệm thu Bài tập Cá nhân 3

- **Mã nguồn & Branch:** Branch `bt3/23001827-vt116-soft-delete-restore` sạch sẽ, không có deliberate mutation, bảo toàn 100% chức năng của nhóm.
- **Trạng thái nộp bài: READY FOR STUDENT REVIEW** (Sinh viên đã hoàn thành mọi kiểm chứng thực tế).

Phụ lục A: Chi tiết Nhật ký Tương tác AI & Chẩn đoán IDE

Phụ lục này ghi lại toàn bộ tiến trình tương tác giữa sinh viên **Nguyễn Việt Anh** và trợ lý **Antigravity AI (Gemini 3.8 Flash)** trong suốt quá trình hoàn thành Bài tập cá nhân 3.

Giai đoạn	Prompt Chính / Thao tác	Đề xuất của AI	Phản biện & Quyết định của Sinh viên
Phase 1 & 2 (Audit & Plan)	Khảo sát repo, lập spec và assumptions cho VT-116.	AI đề xuất: dùng status `cancelled` làm soft delete; giả định có bảng users; soft delete giữ nguyên slot.	Sinh viên tại Checkpoint 1: Bác bỏ hoàn toàn. Bắt buộc nhả slot khi soft delete; ẩn khỏi API đọc (404); lập API trash riêng.
Phase 4 (Test-First)	Viết test trước code (RED), sau đó tạo patch (GREEN).	AI sinh 13 tests (RED fail 12/13), sau đó tạo patch tối thiểu trên 5 files (GREEN pass 92/92).	Sinh viên tại Checkpoint 2: Chưa phê duyệt patch. Chỉ ra lỗ hổng gán owner anonymous, thiếu auth check và race condition.
Checkpoint 2 (Chẩn đoán IDE)	Yêu cầu chẩn đoán vì sao IDE báo đỏ hàng loạt file dù test pass.	AI chạy `pyright`, `compileall`, `ruff` và đối chiếu cấu hình interpreter.	Phát hiện Language Server dùng nhầm System Python 3.14 chưa có thư viện; code thật hoàn toàn 0 lỗi type error. Sửa ruff lint/format.
Phase 5 & 6 (Mutation & Perf)	Thực hiện mutation đảo true/false; load test 40 tasks.	AI đảo `owner == caller` $\rightarrow$ test fail (Killed); viết load test harness đo tải 200 tasks in-flight.	Sinh viên kiểm chứng: Mutation bị tiêu diệt 100%; tải đạt p95 = 482.7 ms (so với $\leq$ 1.5s); lưu toàn bộ log thực nghiệm.